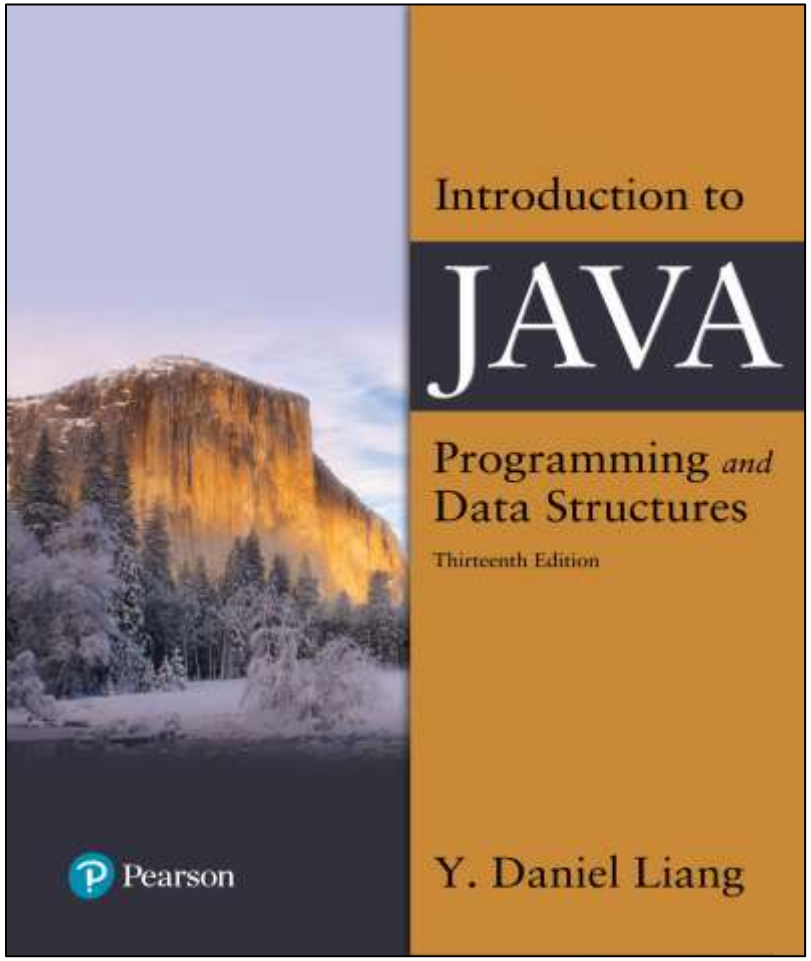
Introduction to Java Programming and Data Structures

Thirteenth Edition



Chapter 3

Selections

Pearson

Copyright © 2024 Pearson Education, Inc. All Rights Reserved

Motivations

If you assigned a negative value for **`radius`** in Listing 2.2, `ComputeAreaWithConsoleInput.java`, the program would print an invalid result. If the radius is negative, you don't want the program to compute the area. How can you deal with this situation?

Pearson

Copyright © 2024 Pearson Education, Inc. All Rights Reserved

Objectives (1 of 2)

3.1 To declare **`boolean`** variables and write Boolean expressions using relational operators (§3.2).

3.2 To implement selection control using one-way **`if`** statements (§3.3).

3.3 To implement selection control using two-way **`if-else`** statements (§3.4).

3.4 To implement selection control using nested **`if`** and multi-way **`if`** statements (§3.5).

3.5 To avoid common errors and pitfalls in **`if`** statements (§3.6).

3.6 To generate random numbers using the **`Math.random()`** method (§3.7).

Pearson

Copyright © 2024 Pearson Education, Inc. All Rights Reserved

Objectives (2 of 2)

3.7 To program using selection statements for a variety of examples (**`SubtractionQuiz`**, **`BMI`**, **`ComputeTax`**) (§§3.7–3.9).

3.8 To combine conditions using logical operators (**`&&`**, **`||`**, and **`!`**) (§3.10).

3.9 To program using selection statements with combined conditions (**`LeapYear`**, **`Lottery`**) (§§3.11–3.12).

3.10 To implement selection control using **`switch`** statements and expressions (§3.13).

3.11 To write expressions using the conditional expression (§3.14).

3.12 To examine the rules governing operator precedence and associativity (§3.15).

3.13 To apply common techniques to debug errors (§3.16).

Pearson

Copyright © 2024 Pearson Education, Inc. All Rights Reserved

The `boolean` Type and Operators

Often in a program you need to compare two values, such as whether i is greater than j. Java provides six comparison operators (also known as relational operators) that can be used to compare two values. The result of the comparison is a Boolean value: true or false.

```
boolean b = (1 > 2);
```

Pearson

Copyright © 2024 Pearson Education, Inc. All Rights Reserved

Relational Operators

Java Operator	Mathematics Symbol	Name	Example (radius is 5)	Result
<	<	less than	radius < 0	false
<=	≤	less than or equal to	radius <= 0	false
>	>	greater than	radius > 0	true
>=	≥	greater than or equal to	radius >= 0	true
==	=	equal to	radius == 0	false
!=	≠	not equal to	radius != 0	true

Pearson

Copyright © 2024 Pearson Education, Inc. All Rights Reserved

Problem: A Simple Math Learning Tool

This example creates a program to let a first grader practice additions. The program randomly generates two single-digit integers number1 and number2 and displays a question such as “What is 7 + 9?” to the student. After the student types the answer, the program displays a message to indicate whether the answer is true or false.

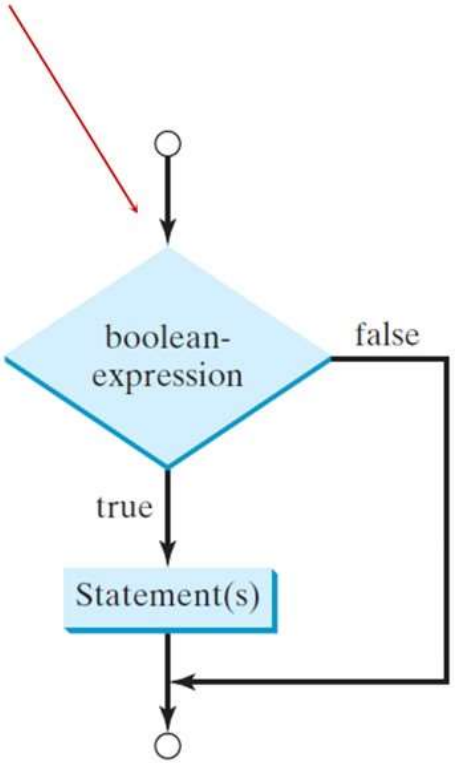
AdditionQuiz

Pearson

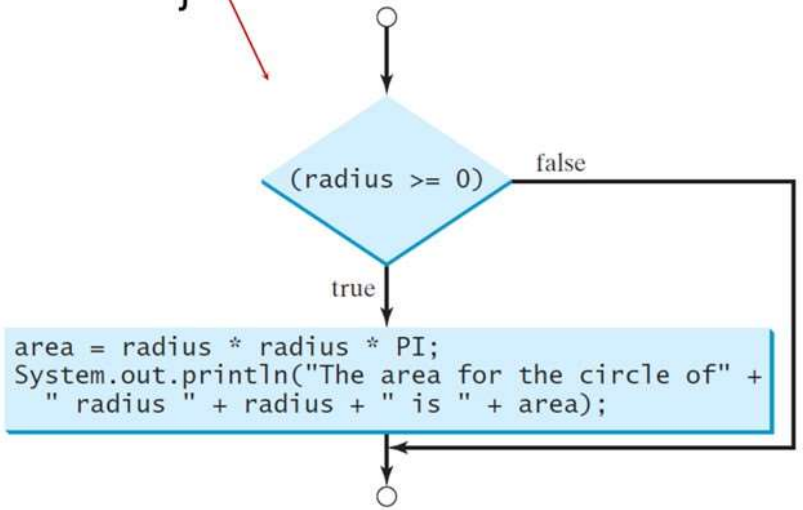
Copyright © 2024 Pearson Education, Inc. All Rights Reserved

One-Way `if` Statements

```
if (boolean-expression) {
  statement(s);
}
```




```
if (radius >= 0) {
  area = radius * radius * PI;
  System.out.println("The area"
    + " for the circle of radius "
    + radius + " is " + area);
}
```



Pearson

Copyright © 2024 Pearson Education, Inc. All Rights Reserved

Note

```
if i > 0 {
  System.out.println("i is positive");
}
```

(a) Wrong

```
if (i > 0) {
  System.out.println("i is positive");
}
```

(b) Correct

```
if (i > 0) {
  System.out.println("i is positive");
}
```

(a)

Equivalent

```
if (i > 0)
  System.out.println("i is positive");
```

(b)

Pearson

Copyright © 2024 Pearson Education, Inc. All Rights Reserved

Simple `if` Demo

Write a program that prompts the user to enter an integer. If the number is a multiple of 5, print HiFive. If the number is divisible by 2, print HiEven.

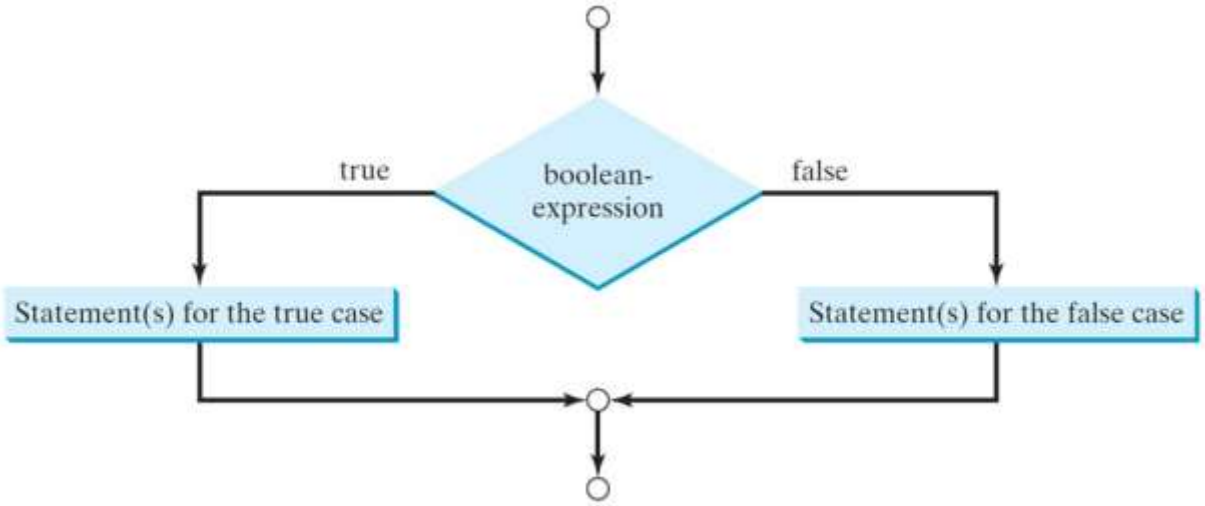
SimpleIfDemo

Pearson

Copyright © 2024 Pearson Education, Inc. All Rights Reserved

The Two-Way `if` Statement

```
if (boolean-expression) {
   statement(s)-for-the-true-case;
}
else {
   statement(s)-for-the-false-case;
}
```



Pearson

Copyright © 2024 Pearson Education, Inc. All Rights Reserved

if-else Example

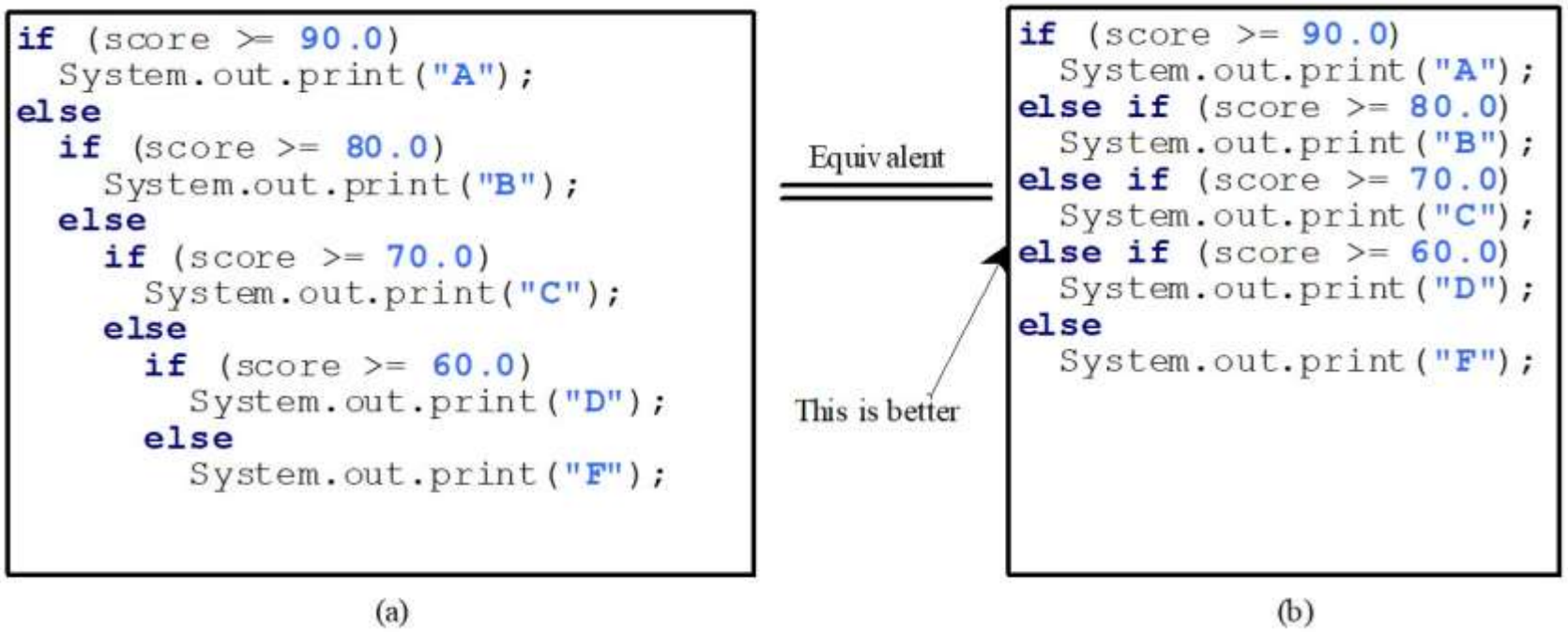
```
if (radius >= 0) {
  area = radius * radius * 3.14159;

    System out.println("The area for the “
    + “circle of radius " + radius +
    " is " + area);
}
else {
  System.out.println("Negative input");
}
```

Pearson

Copyright © 2024 Pearson Education, Inc. All Rights Reserved

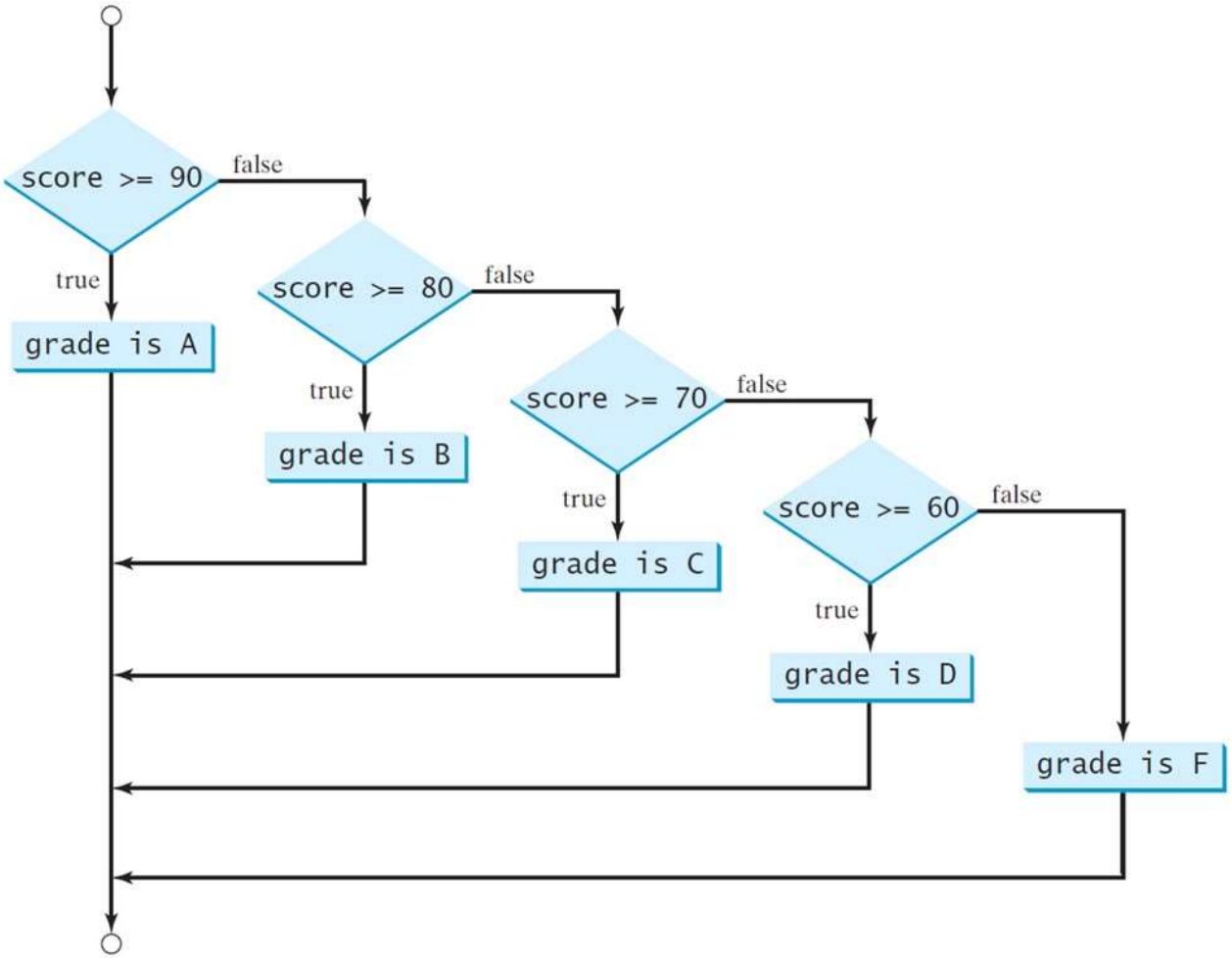
Multiple Alternative `if` Statements



Pearson

Copyright © 2024 Pearson Education, Inc. All Rights Reserved

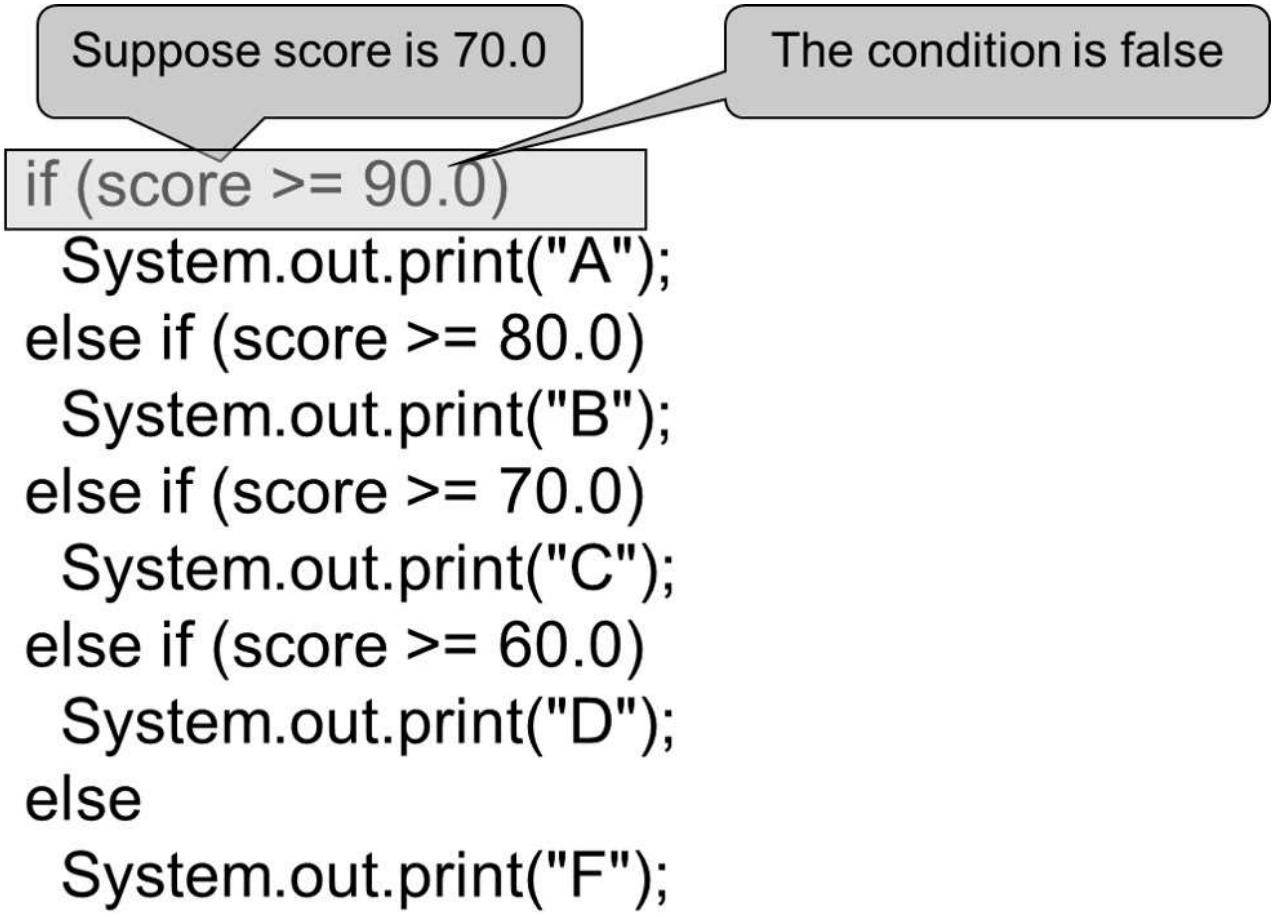
Multi-Way `if-else` Statements



Pearson

Copyright © 2024 Pearson Education, Inc. All Rights Reserved

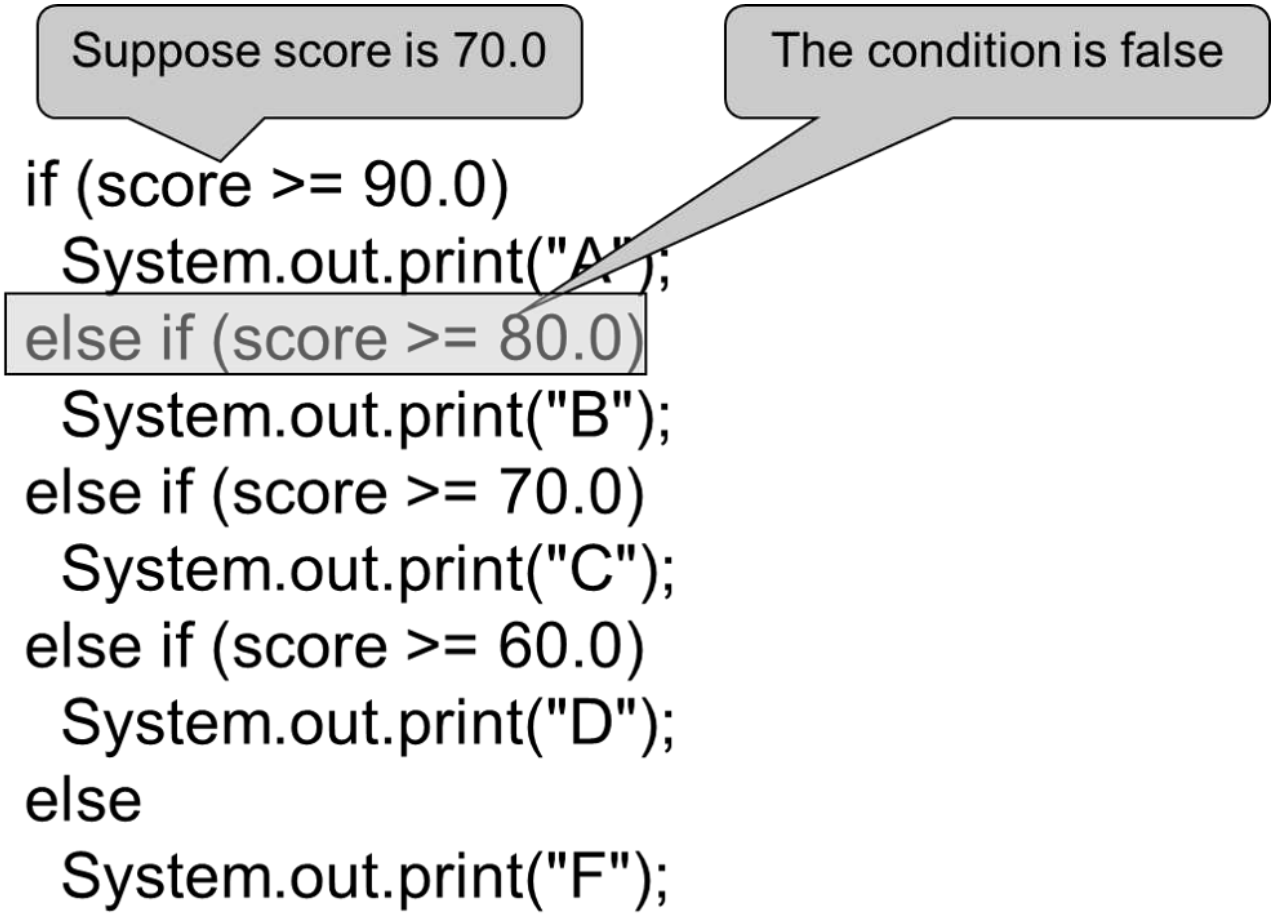
Trace `if-else` Statement (1 of 5)



Pearson

Copyright © 2024 Pearson Education, Inc. All Rights Reserved

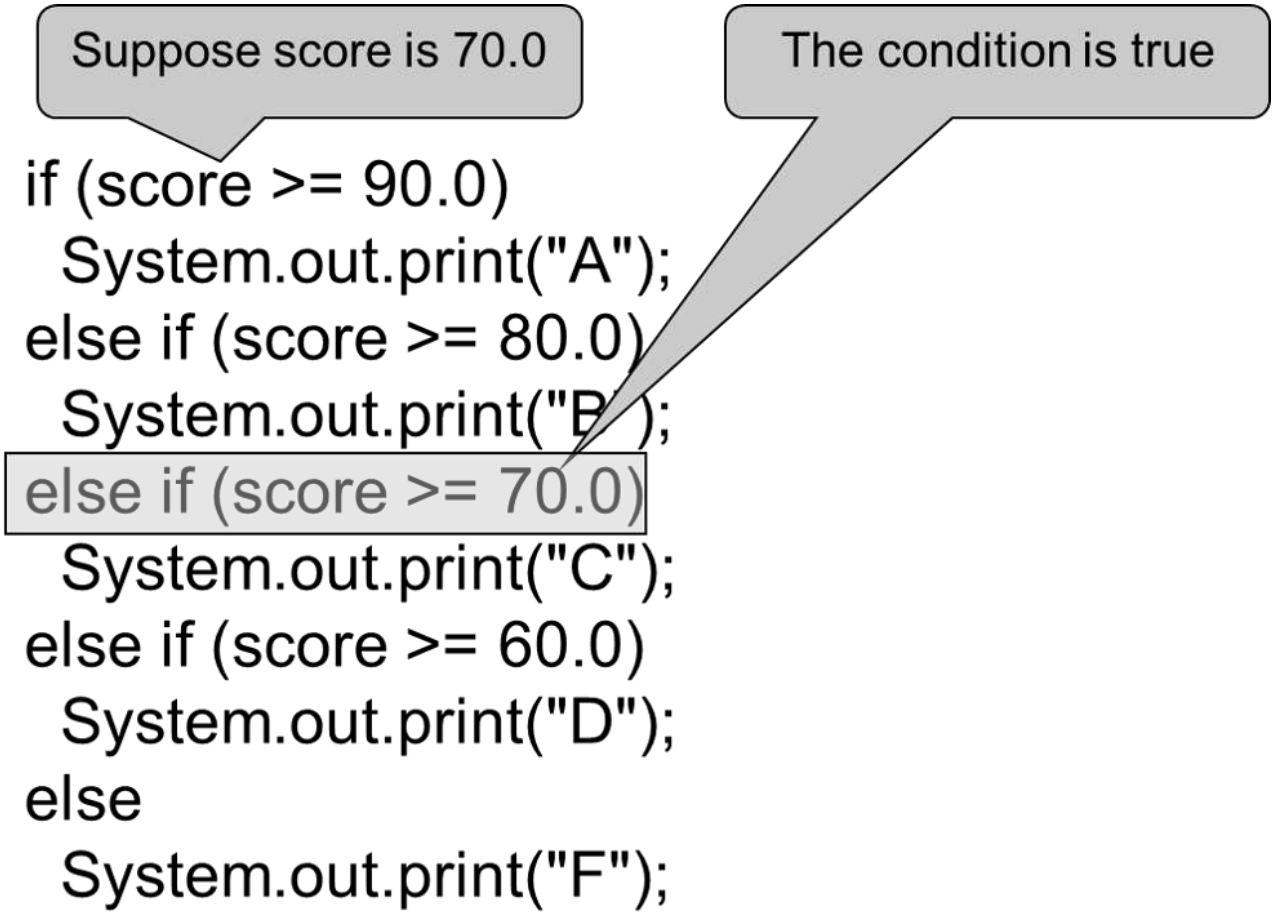
Trace `if-else` Statement (2 of 5)



Pearson

Copyright © 2024 Pearson Education, Inc. All Rights Reserved

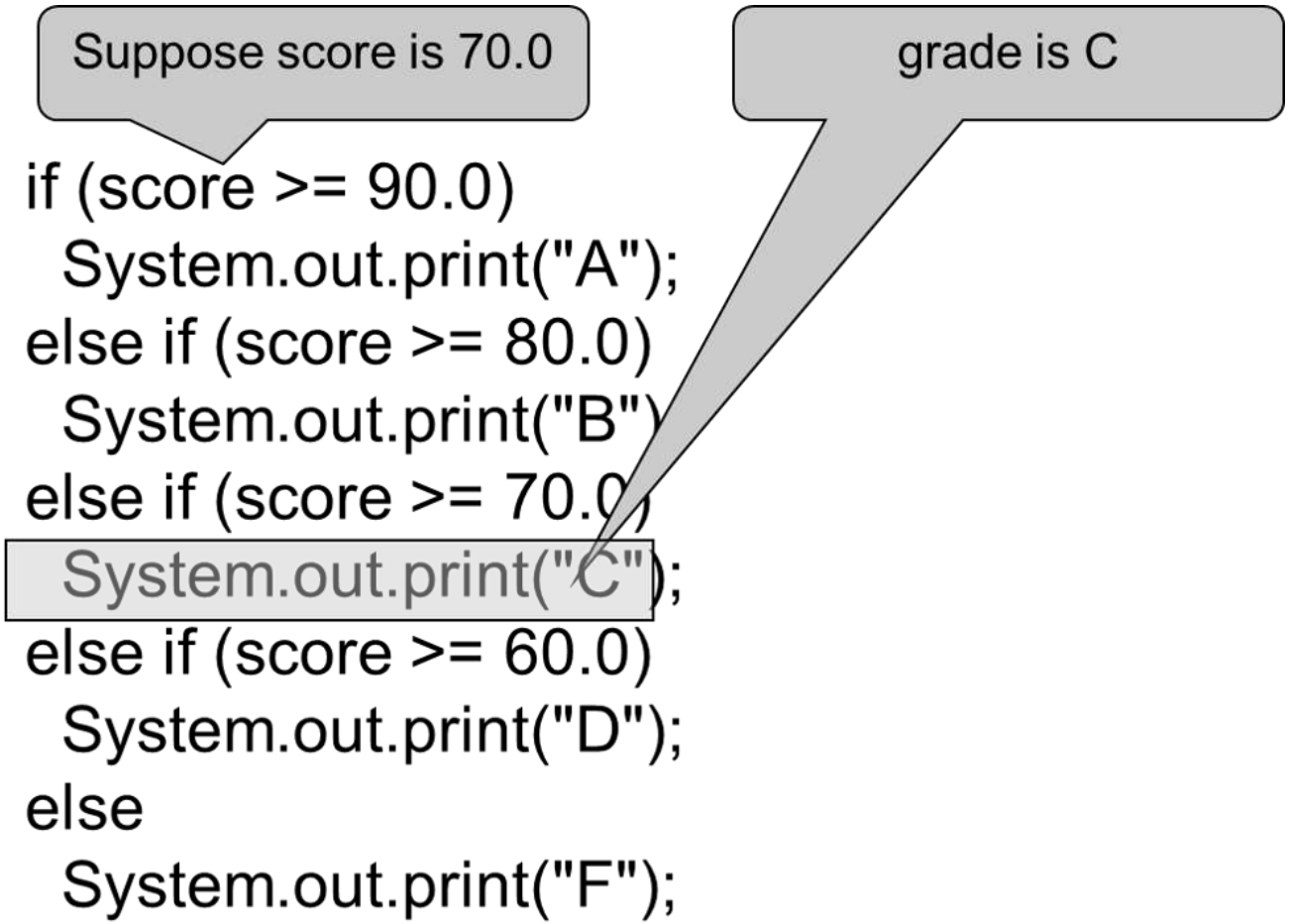
Trace `if-else` Statement (3 of 5)



Pearson

Copyright © 2024 Pearson Education, Inc. All Rights Reserved

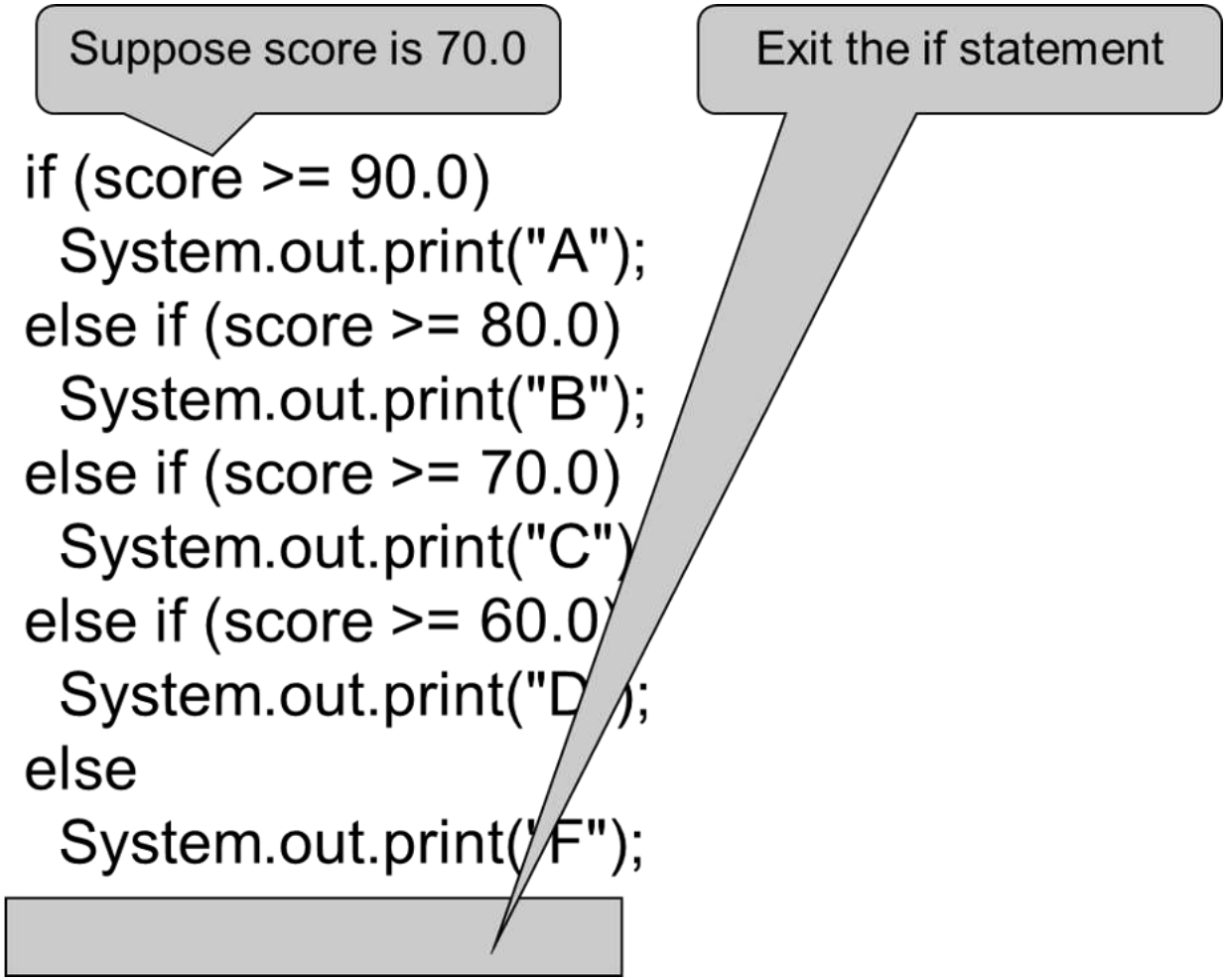
Trace `if-else` Statement (4 of 5)



Pearson

Copyright © 2024 Pearson Education, Inc. All Rights Reserved

Trace `if-else` Statement (5 of 5)

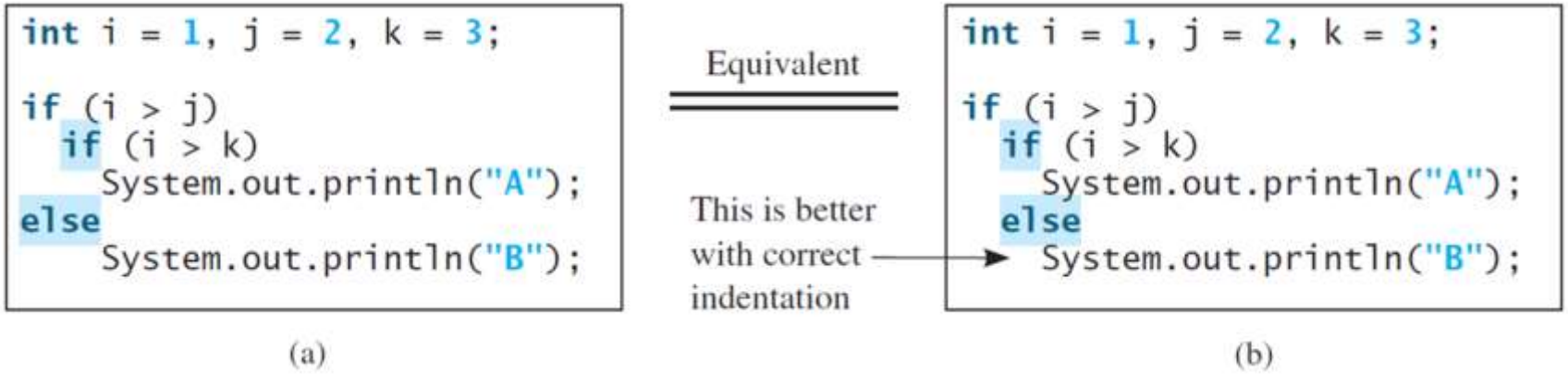


Pearson

Copyright © 2024 Pearson Education, Inc. All Rights Reserved

Note (1 of 2)

The **`else`** clause matches the most recent **`if`** clause in the same block.



Pearson

Copyright © 2024 Pearson Education, Inc. All Rights Reserved

Note (2 of 2)

Nothing is printed from the preceding statement. To force the **`else`** clause to match the first if clause, you must add a pair of braces:

```
int i = 1;
int j = 2;
int k = 3;
if (i > j) {
   if (i > k)
    System.out.println("A");
}
else
   System.out.println("B");
```

This statement prints B.

Pearson

Copyright © 2024 Pearson Education, Inc. All Rights Reserved

Common Errors

Adding a semicolon at the end of an **`if`** clause is a common mistake.

```
if (radius >= 0);
{
  area = radius*radius*PI;
  System.out.println(
    "The area for the circle of radius " +
    radius + " is " + area);
}
```

Wrong

This mistake is hard to find, because it is not a compilation error or a runtime error, it is a logic error.

This error often occurs when you use the next-line block style.

Pearson

Copyright © 2024 Pearson Education, Inc. All Rights Reserved

TIP

```
if (number % 2 == 0)
  even = true;
else
  even = false;
```

(a)

Equivalent

```
boolean even
  = number % 2 == 0;
```

(b)

Pearson

Copyright © 2024 Pearson Education, Inc. All Rights Reserved

Caution

```
if (even == true)
  System.out.println(
    "It is even.");
```

(a)

Equivalent

```
if (even)
  System.out.println(
    "It is even.");
```

(b)

Pearson

Copyright © 2024 Pearson Education, Inc. All Rights Reserved

Problem: An Improved Math Learning Tool

This example creates a program to teach a first grade child how to learn subtractions. The program randomly generates two single-digit integers **`number1`** and **`number2`** with **`number1 >= number2`** and displays a question such as "What is 9 – 2?" to the student. After the student types the answer, the program displays whether the answer is correct.

SubtractionQuiz

Pearson

Copyright © 2024 Pearson Education, Inc. All Rights Reserved

Problem: Body Mass Index

Body Mass Index (BMI) is a measure of health on weight. It can be calculated by taking your weight in kilograms and dividing by the square of your height in meters. The interpretation of BMI for people 16 years or older is as follows:

BMI	Interpretation
BMI < 18.5	Underweight
18.5 <= BMI < 25.0	Normal
25.0 <= BMI < 30.0	Overweight
30.0 <= BMI	Obese

ComputeAndInterpretBMI

Pearson

Copyright © 2024 Pearson Education, Inc. All Rights Reserved

Problem: Computing Taxes (1 of 2)

The US federal personal income tax is calculated based on the filing status and taxable income. There are four filing statuses: single filers, married filing jointly, married filing separately, and head of household. The tax rates for 2009 are shown below.

Marginal Tax Rate	Single	Married Filing Jointly or Qualifying Widow(er)	Married Filing Separately	Head of Household
10%	$0 - $8,350	$0 - $16,700	$0 - $8,350	$0 - $11,950
15%	$8,351 - $33,950	$16,701 - $67,900	$8,351 - $33,950	$11,951 - $45,500
25%	$33,951 - $82,250	$67,901 - $137,050	$33,951 - $68,525	$45,501 - $117,450
28%	$82,251 - $171,550	$137,051 - $208,850	$68,526 - $104,425	$117,451 - $190,200
33%	$171,551 - $372,950	$208,851 - $372,950	$104,426 - $186,475	$190,201 - $372,950
35%	$372,951+	$372,951+	$186,476+	$372,951+

Pearson

Copyright © 2024 Pearson Education, Inc. All Rights Reserved

Problem: Computing Taxes (2 of 2)

```
if (status == 0) {
  // Compute tax for single filers
}
else if (status == 1) {
  // Compute tax for married file jointly
  // or qualifying widow(er)
}
else if (status == 2) {
  // Compute tax for married file separately
}
else if (status == 3) {
  // Compute tax for head of household
}
else {
  // Display wrong status
}
```

ComputeTax

Pearson

Copyright © 2024 Pearson Education, Inc. All Rights Reserved

Logical Operators

Operator	Name	Description
!	not	logical negation
&&	and	logical conjunction
||	or	logical disjunction
^	exclusive or	logical exclusion

Pearson

Copyright © 2024 Pearson Education, Inc. All Rights Reserved

Truth Table for Operator !

p	!p	Example (assume age = 24, weight = 140)
true	false	!(age > 18) is false, because (age > 18) is true.
false	true	!(weight == 150) is true, because (weight == 150) is false.

Pearson

Copyright © 2024 Pearson Education, Inc. All Rights Reserved

Truth Table for Operator &&

p_1	p_2	p_1 && p_2	Example (assume age = 24, weight = 140)
false	false	false	(age <= 18) && (weight < 140) is false, because both conditions are both false.
false	true	false	
true	false	false	(age > 18) && (weight > 140) is false, because (weight > 140) is false.
true	true	true	(age > 18) && (weight >= 140) is true, because both (age > 18) and (weight >= 140) are true.

Pearson

Copyright © 2024 Pearson Education, Inc. All Rights Reserved

Truth Table for Operator ||

p_1	p_2	$p_1 \,\|\|\, p_2$	**Example (assume age = 24, weight = 140)**
false	false	false	
false	true	true	(age > 34) || (weight <= 140) is true, because (age > 34) is false, but (weight <= 140) is true.
true	false	true	(age > 14) || (weight >= 150) is false, because (age > 14) is true.
true	true	true	

Pearson

Copyright © 2024 Pearson Education, Inc. All Rights Reserved

Truth Table for Operator ^

p_1	p_2	p_1 ^ p_2	Example (assume age = 24, weight = 140)
false	false	false	(age > 34) ^ (weight > 140) is true, because (age > 34) is false and (weight > 140) is false.
false	true	true	(age > 34) ^ (weight >= 140) is true, because (age > 34) is false but (weight >= 140) is true.
true	false	true	(age > 14) ^ (weight > 140) is true, because (age > 14) is true and (weight > 140) is false.
true	true	false	

Pearson

Copyright © 2024 Pearson Education, Inc. All Rights Reserved

Examples (1 of 2)

Here is a program that checks whether a number is divisible by **2** and **3**, whether a number is divisible by **2** or **3**, and whether a number is divisible by **2** or **3** but not both:

TestBooleanOperators

Pearson

Copyright © 2024 Pearson Education, Inc. All Rights Reserved

Examples (2 of 2)

```
System.out.println("Is " + number + " divisible by 2 and 3? " +
  ((number % 2 == 0) && (number % 3 == 0)));

System.out.println("Is " + number + " divisible by 2 or 3? " +
  ((number % 2 == 0) || (number % 3 == 0)));

System.out.println("Is " + number +
" divisible by 2 or 3, but not both? " +
  ((number % 2 == 0) ^ (number % 3 == 0)));
```

TestBooleanOperators

Pearson

Copyright © 2024 Pearson Education, Inc. All Rights Reserved

The & and | Operators (1 of 2)

Supplement III.B, “The & and | Operators”

Pearson

Copyright © 2024 Pearson Education, Inc. All Rights Reserved

The & and | Operators (2 of 2)

```
If x is 1, what is x after this expression?

(x > 1) & (x++ < 10)


If x is 1, what is x after this expression?

(1 > x) && ( 1 > x++)


How about (1 == x) | (10 > x++)?

(1 == x) || (10 > x++)?
```

Pearson

Copyright © 2024 Pearson Education, Inc. All Rights Reserved

Problem: Determining Leap Year?

This program first prompts the user to enter a year as an int value and checks if it is a leap year.

A year is a leap year if it is divisible by 4 but not by 100, or it is divisible by 400.

```
(year % 4 == 0 && year % 100 != 0) || (year % 400 == 0)
```

LeapYear

Pearson

Copyright © 2024 Pearson Education, Inc. All Rights Reserved

Problem: Lottery

Write a program that randomly generates a lottery of a two-digit number, prompts the user to enter a two-digit number, and determines whether the user wins according to the following rule:

- If the user input matches the lottery in exact order, the award is $10,000.
- If the user input matches the lottery, the award is $3,000.
- If one digit in the user input matches a digit in the lottery, the award is $1,000.

Lottery

Pearson

Copyright © 2024 Pearson Education, Inc. All Rights Reserved

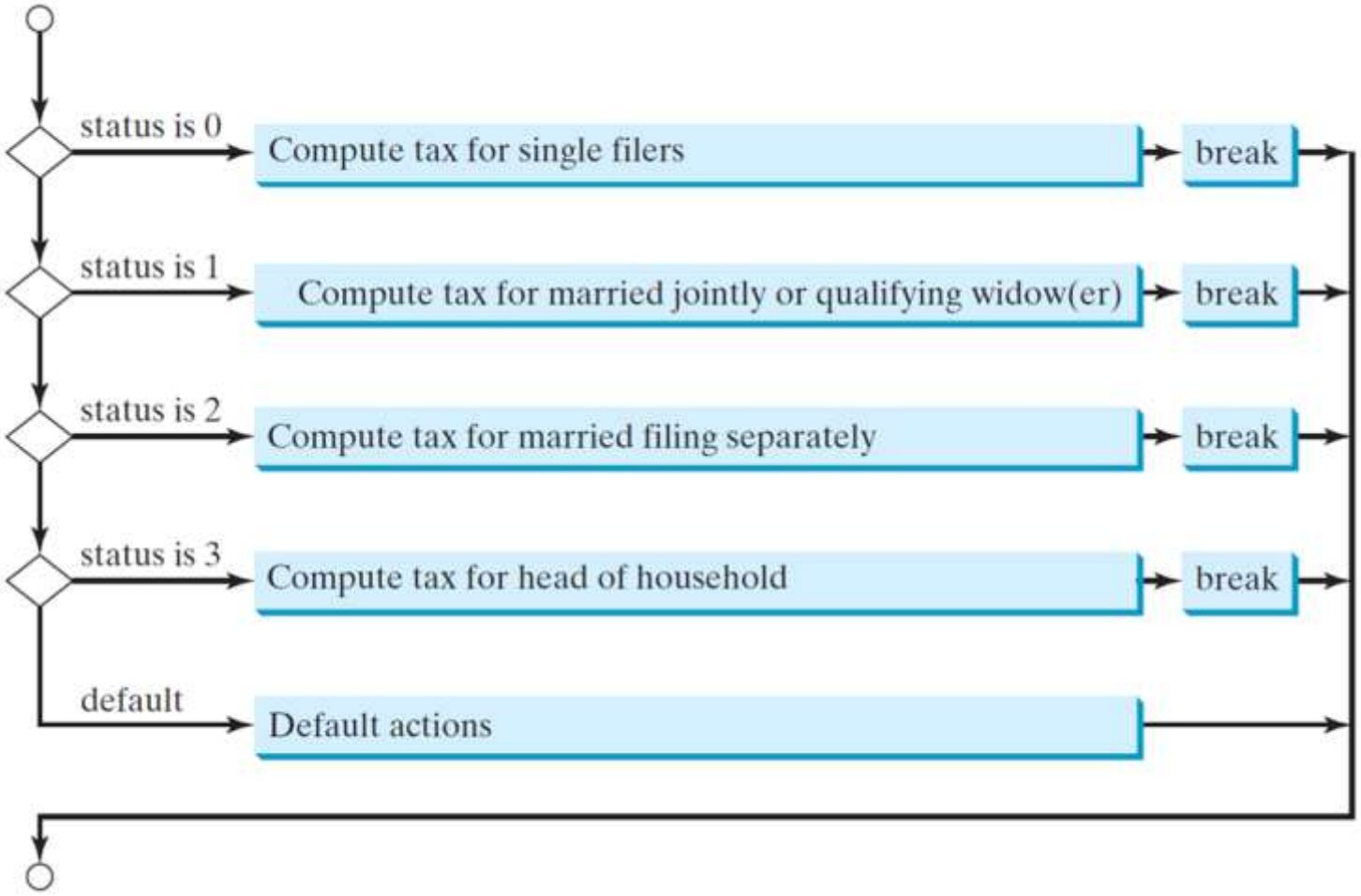
`switch` Statements

```
switch (status) {
   case 0: compute taxes for single filers;
           break;
   case 1: compute taxes for married file jointly;
           break;
   case 2: compute taxes for married file separately;
           break;
   case 3: compute taxes for head of household;
           break;
   default: System.out.println("Errors: invalid status");
           System.exit(1);
}
```

Pearson

Copyright © 2024 Pearson Education, Inc. All Rights Reserved

`switch` Statement Flow Chart



Pearson

Copyright © 2024 Pearson Education, Inc. All Rights Reserved

`switch` Statement Rules (1 of 2)

The switch-expression must yield a value of char, byte, short, or int type and must always be enclosed in parentheses.

The value1, ..., and valueN must have the same data type as the value of the switch-expression. The resulting statements in the case statement are executed when the value in the case statement matches the value of the switch-expression. Note that value1, ..., and valueN are constant expressions, meaning that they cannot contain variables in the expression, such as 1 + x.

```
switch (switch-expression) {
  case value1:  statement(s)1;
                break;
  case value2: statement(s)2;
                break;
  ...
  case valueN: statement(s)N;
                break;
  default: statement(s)-for-default;
}
```

Pearson

Copyright © 2024 Pearson Education, Inc. All Rights Reserved

`switch` Statement Rules (2 of 2)

The keyword break is optional, but it should be used at the end of each case in order to terminate the remainder of the switch statement. If the break statement is not present, the next case statement will be executed.

The default case, which is optional, can be used to perform actions when none of the specified cases matches the switch-expression.

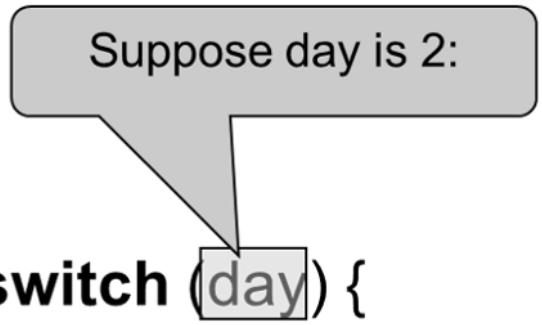
```
switch (switch-expression) {
  case value1:  statement(s)1;
        break;
  case value2: statement(s)2;
        break;
  …
  case valueN: statement(s)N;
        break;
  default: statement(s)-for-default;
}
```

When the value in a **case** statement matches the value of the **switch-expression**, the statements *starting from this case* are executed until either a **break** statement or the end of the **switch** statement is reached.

Pearson

Copyright © 2024 Pearson Education, Inc. All Rights Reserved

Trace `switch` Statement (1 of 7)

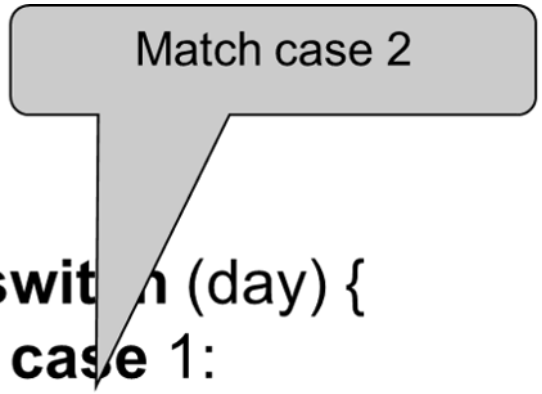



```
switch (day) {
  case 1:
  case 2:
  case 3:
  case 4:
  case 5: System.out.println("Weekday"); break;
  case 0:
  case 6: System.out.println("Weekend");
}
```

Pearson

Copyright © 2024 Pearson Education, Inc. All Rights Reserved

Trace `switch` Statement (2 of 7)

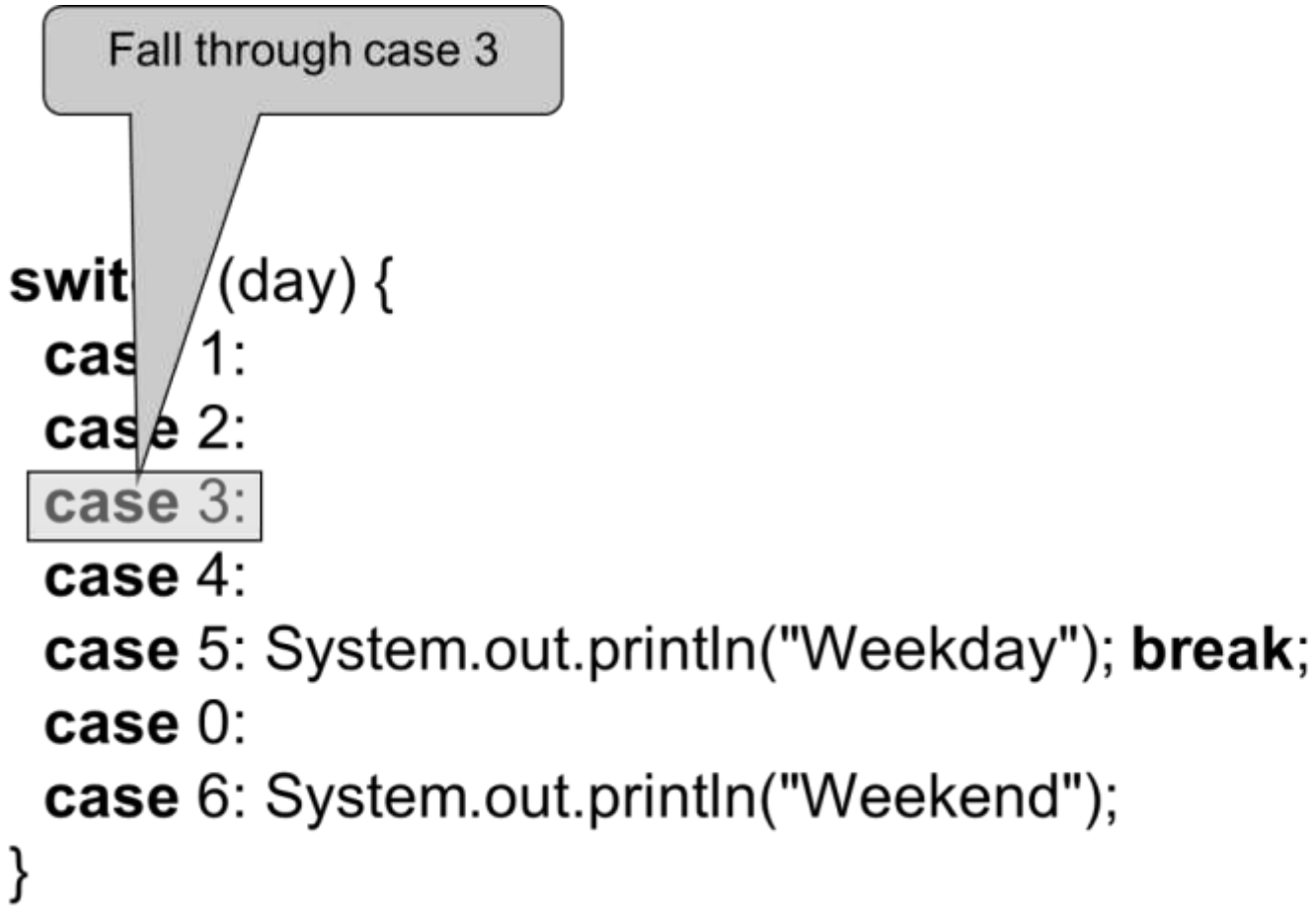



```
switch (day) {
  case 1:
  case 2:
  case 3:
  case 4:
  case 5: System.out.println("Weekday"); break;
  case 0:
  case 6: System.out.println("Weekend");
}
```

Pearson

Copyright © 2024 Pearson Education, Inc. All Rights Reserved

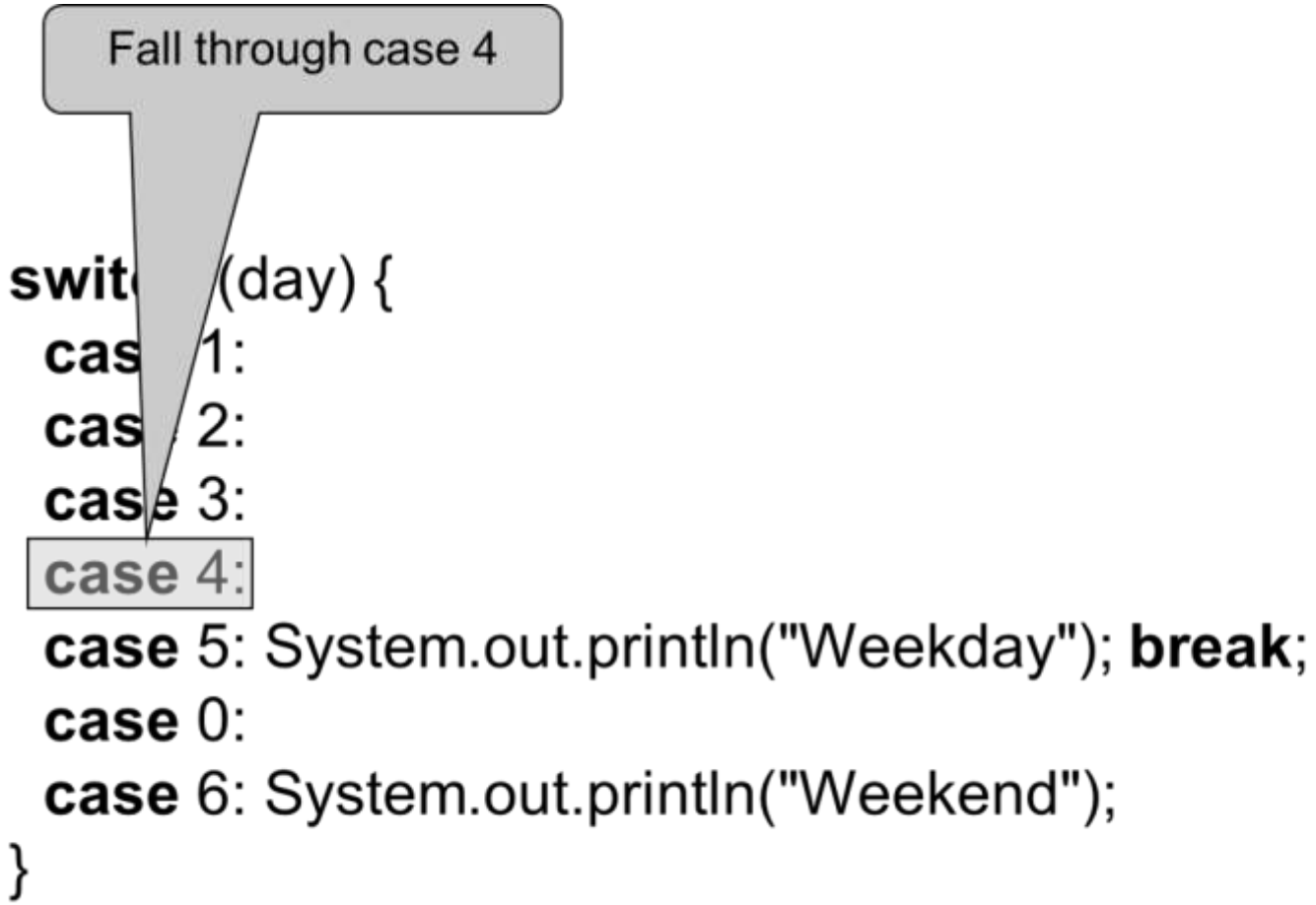
Trace `switch` Statement (3 of 7)



Pearson

Copyright © 2024 Pearson Education, Inc. All Rights Reserved

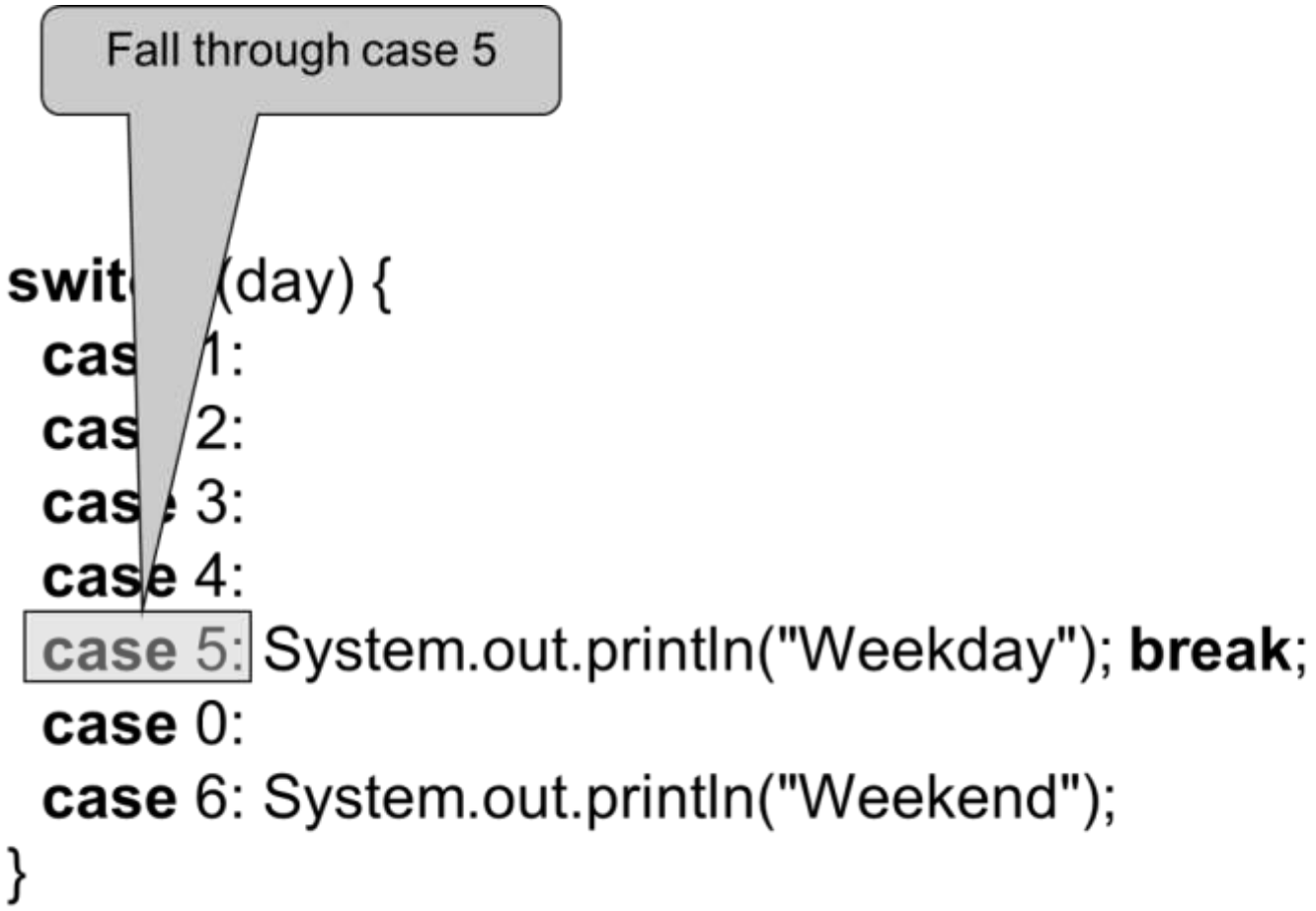
Trace `switch` Statement (4 of 7)



Pearson

Copyright © 2024 Pearson Education, Inc. All Rights Reserved

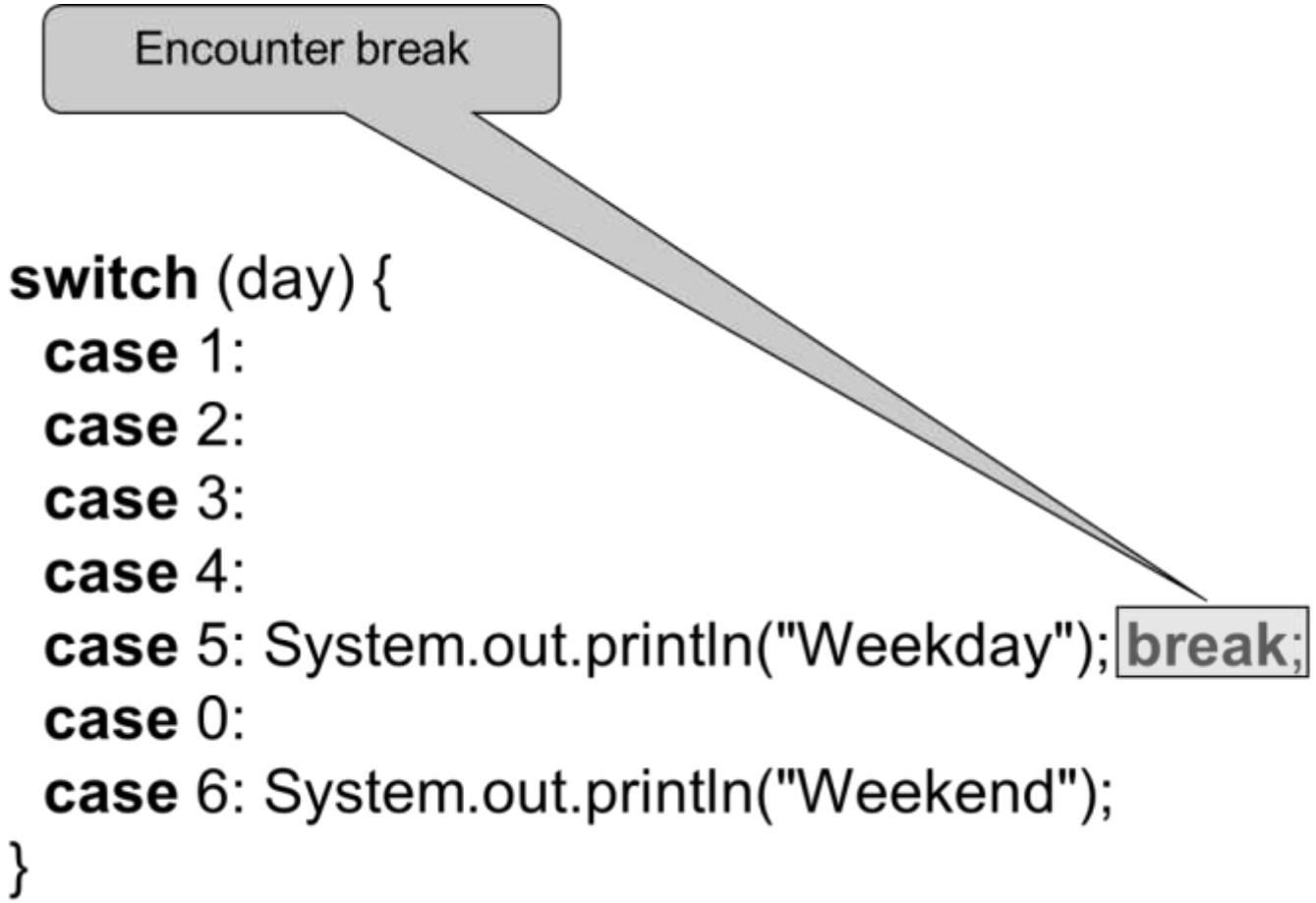
Trace `switch` Statement (5 of 7)



Pearson

Copyright © 2024 Pearson Education, Inc. All Rights Reserved

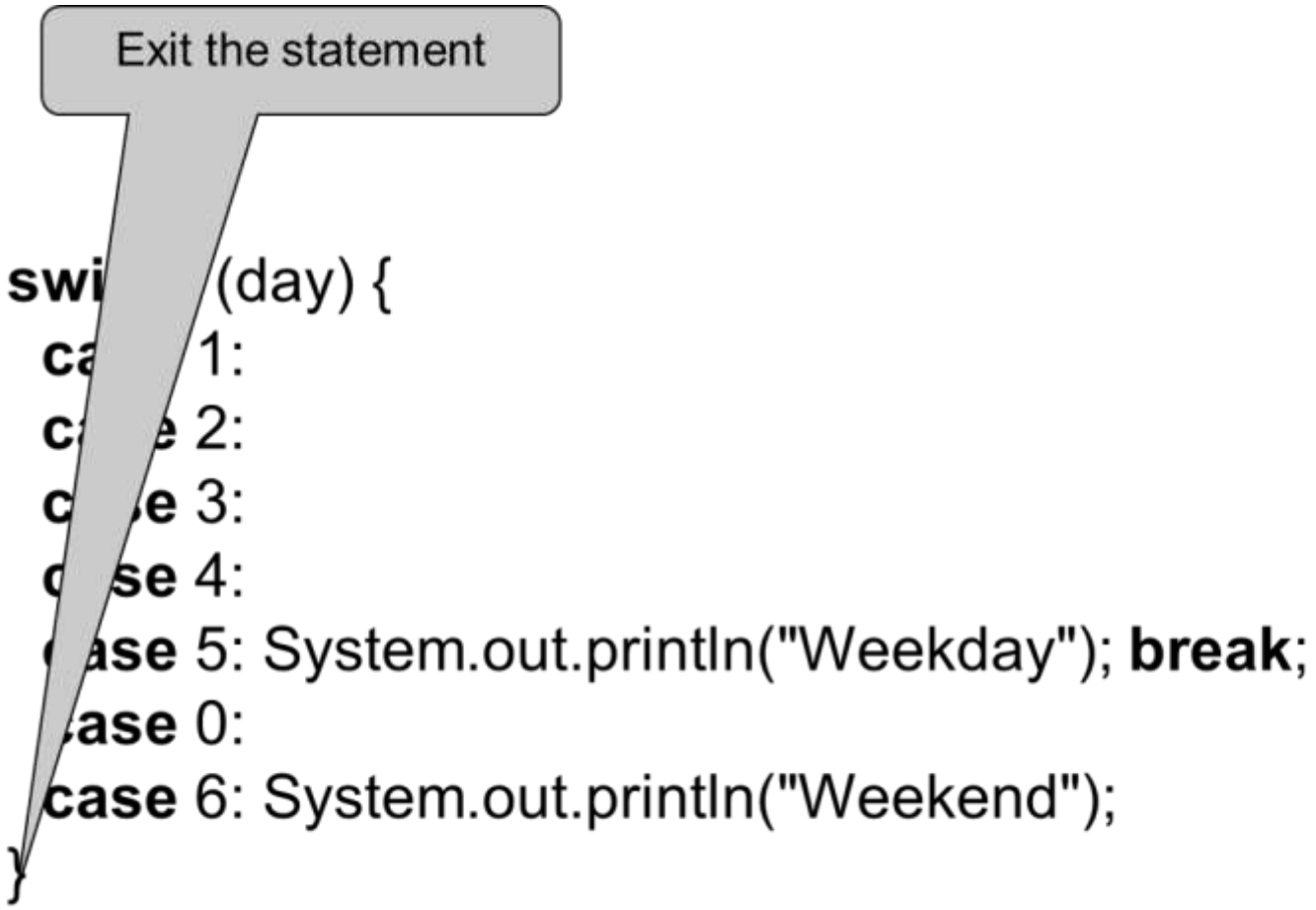
Trace `switch` Statement (6 of 7)



Pearson

Copyright © 2024 Pearson Education, Inc. All Rights Reserved

Trace `switch` Statement (7 of 7)



Pearson

Copyright © 2024 Pearson Education, Inc. All Rights Reserved

Problem: Chinese Zodiac

Write a program that prompts the user to enter a year and displays the animal for the year.



year % 12 =
0: monkey
1: rooster
2: dog
3: pig
4: rat
5: ox
6: tiger
7: rabbit
8: dragon
9: snake
10: horse
11: sheep

ChineseZodiac

Pearson

Copyright © 2024 Pearson Education, Inc. All Rights Reserved

JDK 14 Arrow Operator (->)

Forgetting a break statement when it is needed is a common error. To avoid this type of errors, JDK 14 introduced a new arrow operator (->). You can use the arrow operator to replace the colon operator (:). With the arrow operator, there is no need to use the break statement. When a case is matched, the matching statement(s) are executed, and the switch statement is finished.

```
int day = 1;
switch (day) {
  case 1 -> System.out.print(1 + " ");
  case 2 -> System.out.print(2 + " ");
  case 3 -> System.out.print(3 + " ");
}
```

Pearson

Copyright © 2024 Pearson Education, Inc. All Rights Reserved

JDK 14 `switch` Expression

Java 14 also introduced switch expressions. A switch expression returns a value. Here is an example:

```
int day = 1;
    System.out.println(
      switch (day) {
        case 1 -> 1 + " ";
        case 2 -> 2 + " ";
        case 3 -> 3 + " ";
        default -> " ";
      }
);
```

The switch expression in this example returns a string. A switch expression must cover all cases, while a switch statement does not have to cover all cases. In the preceding example, the default clause is to required to cover the integers not listed in the cases.

Pearson

Copyright © 2024 Pearson Education, Inc. All Rights Reserved

JDK 14 `switch` Combining Cases

In Java 14, the cases can be combined. For example, the preceding code can be simplified as follows:

```
int day = 1;
System.out.println(
  switch (day) {
    case 1, 2, 3 -> day + " ";
    default -> " ";
  }
);
```

case 1, 2, 3 means case 1, case 2, or case 3.

Pearson

Copyright © 2024 Pearson Education, Inc. All Rights Reserved

The `yield` Keyword (1 of 2)

In Java 14, the cases can be combined. For example, the preceding code can be simplified as follows:

```
int day = 1;
System.out.println(
  switch (day) {
    case 1, 2, 3 -> day + " ";
    default -> " ";
  }
);
```

case 1, 2, 3 means case 1, case 2, or case 3.

Pearson

Copyright © 2024 Pearson Education, Inc. All Rights Reserved

The `yield` Keyword (2 of 2)

If the result for a matching case in a switch expression is not a simple value, you need to use the yield keyword to return the value. Here is an example,

```
int year = 2000;
   int month = 2;
   System.out.println(
     switch (month) {
       case 2 -> {
         if (isLeapYear(year))
           yield "29 days";
         else
           yield "28 days";
       }
       default -> " ";
     }
   );
```

Pearson

Copyright © 2024 Pearson Education, Inc. All Rights Reserved

Conditional Operators

```
if (x > 0)
  y = 1
else
  y = -1;
```

is equivalent to

```
y = (x > 0) ? 1 : -1;
```

```
(boolean-expression) ? expression1 : expression2
```

Ternary operator

Binary operator

Unary operator

Pearson

Copyright © 2024 Pearson Education, Inc. All Rights Reserved

Conditional Operator (1 of 2)

```
if (num % 2 == 0)
  System.out.println(num + "is even");
else
  System.out.println(num + "is odd");


System.out.println(
  (num % 2 == 0)? num + "is even" :
  num + "is odd");
```

Pearson

Copyright © 2024 Pearson Education, Inc. All Rights Reserved

Conditional Operator (2 of 2)

```
boolean-expression ? exp1 : exp2
```


Copyright © 2024 Pearson Education, Inc. All Rights Reserved

Operator Precedence

- var++, var--
- +, - (Unary plus and minus), ++var,--var
- (type) Casting
- ! (Not)
- *, /, % (Multiplication, division, and remainder)
- +, - (Binary addition and subtraction)
- <, <=, >, >= (Relational operators)
- ==, !=; (Equality)
- ^ (Exclusive OR)
- && (Conditional AND) Short-circuit AND
- || (Conditional OR) Short-circuit OR
- =, +=, -=, *=, /=, %= (Assignment operator)

Pearson

Copyright © 2024 Pearson Education, Inc. All Rights Reserved

Operator Precedence and Associativity

The expression in the parentheses is evaluated first. (Parentheses can be nested, in which case the expression in the inner parentheses is executed first.) When evaluating an expression without parentheses, the operators are applied according to the precedence rule and the associativity rule.

If operators with the same precedence are next to each other, their associativity determines the order of evaluation. All binary operators except assignment operators are left-associative.

Pearson

Copyright © 2024 Pearson Education, Inc. All Rights Reserved

Operator Associativity

When two operators with the same precedence are evaluated, the **associativity** of the operators determines the order of evaluation. All binary operators except assignment operators are **left-associative**.

$a - b + c - d$ is equivalent to $((a - b) + c) - d$

Assignment operators are **right-associative**. Therefore, the expression

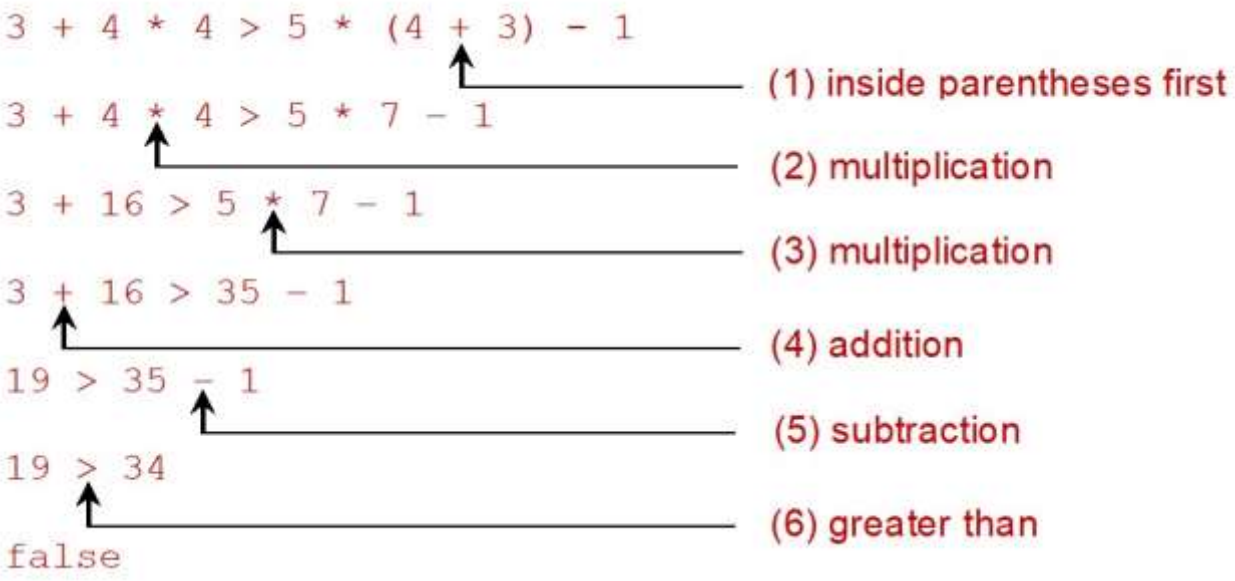
$a = b += c = 5$ is equivalent to $a = (b += (c = 5))$

Pearson

Copyright © 2024 Pearson Education, Inc. All Rights Reserved

Example

Applying the operator precedence and associativity rule, the expression $3 + 4 * 4 > 5 * (4 + 3) - 1$ is evaluated as follows:



Pearson

Copyright © 2024 Pearson Education, Inc. All Rights Reserved

Operand Evaluation Order

Supplement III.A, “Advanced discussions on how an expression is evaluated in the JVM.”

Copyright © 2024 Pearson Education, Inc. All Rights Reserved

Debugging

Logic errors are called **bugs**. The process of finding and correcting errors is called debugging. A common approach to debugging is to use a combination of methods to narrow down to the part of the program where the bug is located. You can hand-trace the program (i.e., catch errors by reading the program), or you can insert print statements in order to show the values of the variables or the execution flow of the program. This approach might work for a short, simple program. But for a large, complex program, the most effective approach for debugging is to use a debugger utility.

Pearson

Copyright © 2024 Pearson Education, Inc. All Rights Reserved

Debugger

Debugger is a program that facilitates debugging. You can use a debugger to

- Execute a single statement at a time.
- Trace into or stepping over a method.
- Set breakpoints.
- Display variables.
- Display call stack.
- Modify variables.

Pearson

Copyright © 2024 Pearson Education, Inc. All Rights Reserved

Debugging in NetBeans

Supplement II.E, Learning Java Effectively with NetBeans

Copyright © 2024 Pearson Education, Inc. All Rights Reserved

Debugging in Eclipse

Supplement II.G, Learning Java Effectively with Eclipse

Pearson

Copyright © 2024 Pearson Education, Inc. All Rights Reserved

Copyright



This work is protected by United States copyright laws and is provided solely for the use of instructors in teaching their courses and assessing student learning. Dissemination or sale of any part of this work (including on the World Wide Web) will destroy the integrity of the work and is not permitted. The work and materials from it should never be made available to students except by instructors using the accompanying text in their classes. All recipients of this work are expected to abide by these restrictions and to honor the intended pedagogical purposes and the needs of other instructors who rely on these materials.

Pearson

Copyright © 2024 Pearson Education, Inc. All Rights Reserved